Chapter 6. Working with Key/Value Data

Like any good distributed computing tool, Spark relies heavily on the key/value pair paradigm to define and parallelize operations, particularly wide transformations that require the data to be redistributed between machines. Anytime we want to perform grouped operations in parallel or change the ordering of records amongst machines—be it computing an aggregation statistic or merging customer records—the key/value functionality of Spark is useful as it allows us to easily parallelize our work. Spark has its own `PairRDDFunctions` class containing operations defined on RDDs of tuples. The `PairRDDFunctions` class, made available through implicit conversion, contains most of Spark's methods for joins, and custom aggregations. The `OrderedRDDFunctions` class contains the methods for sorting. The `OrderedRDDFunctions` are available to RDDs of tuples in which the first element (the key) has an implicit ordering.

> **NOTE**
>
> Similar operations are available on `Dataset`s as discussed in "Grouped Operations on Datasets".

Despite their utility, key/value operations can lead to a number of performance issues. In fact, most expensive operations in Spark fit into the key/value pair paradigm because most wide transformations are key/value

transformations, and most require some fine tuning and care to be performant. These performance considerations will be the focus of this chapter. We hope to provide not just a guide to using the functions in the `PairRDDFunctions` and `OrderedRDDFunctions` classes, but to build on the architecture lessons of Chapters 2 and 5 to present a thorough guide to thinking about how Spark evaluates wide transformations and how to redesign the logic of a program to make tasks that require ordering data most efficient.

In particular, operations on key/value pairs can cause:

- Out-of-memory errors in the driver
- Out-of-memory errors on the executor nodes
- Shuffle failures
- "Straggler tasks" or partitions, which are especially slow to compute

The first problem, memory errors in the driver, is usually caused by actions. We will discuss the performance problems associated with actions on key/value pairs in "Actions on Key/Value Pairs". The last three performance issues—out of memory on the executors, shuffles, and straggler tasks—are all most often caused by shuffles associated with the wide transformations in the `PairRDDFunctions` and `OrderedRDDFunctions` classes. Throughout this chapter, we will focus on two primary techniques to avoid performance problems associated with shuffles, which we call "shuffle less" and "shuffle better":

Shuffle less often

> We will provide techniques to minimize the number of shuffles needed to complete a complex computation. One way to minimize the number of shuffles in a computation that requires several transformations is to

make sure to preserve partitioning across narrow transformations to avoid reshuffling data (see "Preserving Partitioning Information Across Transformations"). In some instances, we can use the same partitioner on a sequence of wide transformations. This can be particularly useful to avoid shuffles during joins and to reduce the number of shuffles required to compute a sequence of wide transformations (see "Co-Grouping" and "Leveraging Co-Located and Co-Partitioned RDDs"). We will also discuss leveraging custom partitioners (see "Custom Partitioning") to distribute the data most effectively for downstream computations as well as how to push computational work into the shuffle stage to make a complicated computation more efficient (see "Secondary Sort and repartitionAndSortWithinPartitions").

Shuffle better

Sometimes, computation cannot be completed without a shuffle. However, not all wide transformations and not all shuffles are equally expensive or prone to failure. By using wide transformations such as `reduceByKey` and `aggregateByKey` that can preform map-side reductions and that do not require loading all the records for one key into memory, you can prevent memory errors on the executors and speed up wide transformations, particularly for aggregation operations (see "What's So Dangerous About the groupByKey Function" and "Preventing out-of-memory errors with aggregation operations"). Lastly, shuffling data in which records are distributed evenly throughout the keys, and which contain a high number of distinct keys, prevents out-of-memory errors on the executors and "straggler tasks" (see "Straggler Detection and Unbalanced Data").

The Goldilocks Example

Throughout this chapter, we will refer to a project that the authors worked on that required finding arbitrary rank statistics in high-dimensionality and high-volume data as an example of a complex key/value transformation.

The client—we will call her Goldilocks—had data representing thousands of different metrics for hundreds of millions of pandas. Her data looked something like Table 6-1.

Table 6-1. Goldilocks example data

Panda name	Happiness	Niceness	Softness	Sweetness
Mama Panda	15.0	0.25	2467.0	0.0
Papa Panda	2.0	1000	35.4	0.0
Baby Panda	10.0	2.0	50.0	0.0
Baby Panda's toy Panda	3.0	8.5	0.2	98.0

The attributes for each panda are represented as a doubles.

Goldilocks wanted us to design an application that would let her input an arbitrary list of integers `n1…nk` and return the `nth` best element in each column. For example, if Goldilocks input 8, 1000, and 20 million, our function would need to return the 8th, 1000th, and 20 millionth best-ranking panda for each attribute column.

To illustrate this example, suppose that Goldilocks wanted to find the 2nd and 4th element from Table 6-1. We would want our function to return something like Table 6-2.

Table 6-2. Goldilocks example result

Column name	Column index	Rank statistics
happiness	1	List(3.0, 15.0)
niceness	2	List(2.0, 1000.0)

softness	3	List(35.4, 2467.0)
sweetness	4	List(0.0, 98.0)

We call this candidate “Goldilocks” because she was very picky and her house (i.e., in-house cluster) was crowded with other users (bears). In this case, Goldilocks would not accept approximate quantile boundaries, but required the output of our function to be values in the original dataset. Thus, this task is inherently expensive since it requires sorting all the values in each column in some way.

Because the data is columnar, we could consider Spark SQL to solve this problem. However, early Spark SQL did not have any support for rank statistics. It may be possible to write a UDF/UDAF to solve the problem, but it would be quite cumbersome because our use case is complicated and cannot be computed on each row. Thus, our solution has to leverage Spark Core.[1]

Goldilocks Version 0: Iterative Solution

One intuitive solution to the Goldilocks problem is to loop through each column, mapping each row to a single value, then use Spark’s `sortBy` and `zipWithIndex` function on each column, and then filter for the indices that correspond to the desired rank statistics.

> **NOTE**
>
> For simplicity, we will assume that the columnar data was read in from stable storage as a `DataFrame`, that the rows are all well formed, and that the string column with each panda’s name was dropped. Consequently, our function takes a `DataFrame` of all double columns (representing the panda data) and a list of `long` types representing the positions of the elements to find for each column (e.g., 1st, 100th). The function should return a map from the column index to a list of the rank statistics in that

column.

Example 6-1 is an implementation of this first solution to the Goldilocks problem in which we loop through each column and sort each one using Spark's distributed sort.

Example 6-1. Goldilocks version 0, iterative solution

```
  def findRankStatistics(
    dataFrame: DataFrame,
    ranks: List[Long]): Map[Int, Iterable[Double]] = {
    require(ranks.forall(_ > 0))
    val numberOfColumns = dataFrame.schema.length
    var i = 0
    var  result = Map[Int, Iterable[Double]]()

    while(i < numberOfColumns){
      val col = dataFrame.rdd.map(row => row.getDouble(i))
      val sortedCol : RDD[(Double, Long)] = col.sortBy(v =>
v).zipWithIndex()
      val ranksOnly = sortedCol.filter{
        //rank statistics are indexed from one. e.g. first element
is 0
        case (colValue, index) =>  ranks.contains(index + 1)
      }.keys
      val list = ranksOnly.collect()
       result += (i -> list)
       i+=1
    }
    result
  }
```

This solution works and is relatively robust, but it is very slow since it has to sort the data once for each column and does so iteratively. In other words, if we have 8,000 columns we have to do 8,000 sorts!

So how can we do better?

Since each sort can be done without knowledge of the other sorts, our

intuition should be that it is possible to parallelize this computation using each column as the unit of parallelization. We can represent the data as one long list of key/value pairs where the keys represent the column indices. Then, we can perform our computation in parallel for each key.

We would map the data in Table 6-1 to the following list of key/value pairs:

(key, value)
(1, 15.0)
(2, 0.25)
(3, 2467.0)
(4, 0.0)
(1, 2.0)
(2, 1000.0)
(3, 35.4)
(4, 0.0)
(1, 10.0)
(2, 2.0)
(3, 50.0)
(4, 0.0)
(1, 3.0)
(2, 8.5)
(3, 0.2)
(4, 98.0)

If we read in our data as a `DataFrame`, we can do this mapping with a simple function like the one shown in Example 6-2.

Example 6-2. Goldilocks version 1, mapping to column index/value pairs

```
  def mapToKeyValuePairs(dataFrame: DataFrame): RDD[(Int, Double)]
= {
    val rowLength = dataFrame.schema.length
    dataFrame.rdd.flatMap(
      row => Range(0, rowLength).map(i => (i, row.getDouble(i)))
    )
  }
```

> **TIP**
>
> Spark's `flatMap` operation mimics the behavior of the `flatMap` operation that is defined on iterators and collections in Scala. `flatMap` is a very versatile narrow transformation, but for those new to Scala it can be a bit confusing. `flatMap` lets the user define a mapping from each record to a collection of elements and then combines the resulting collections together. In this case the mapping is defined from a Spark SQL `Row` object to a sequence of elements, in this case `(columnIndex, value)` pairs. The resulting RDD will have more records than the previous RDD, and each will be of `(columnIndex, value)` pairs. Increasing the total number of records is not a requirement for the `flatMap` operation. In fact, `flatMap` can be particularly useful because unlike `map` it allows us to return an empty collection for one of the records. Thus, the operator can be used to both filter and transform the elements in one pass. In other words, a `map` and `filter` on the same RDD or collection can always be combined into one `flatMap` step.

After applying this function, we can perform this computation in parallel by column index (in this case column index is the key for each record). Framed in this way, the Goldilocks problem is a key/value pair problem. Specifically:

Design a function that takes an input RDD of integer/double pairs and a

list of longs, `n1` … `nk` and returns a map of key to a list of `k` doubles that are the n1th, n2th .. nkth elements for that column index (key).

How to Use PairRDDFunctions and OrderedRDDFunctions

If you have been using Spark for a while, you are probably familiar with the `PairRDDFunctions` and `OrderedRDDFunctions` classes. However, we will still provide a brief introduction about how to use them. If you are new to these functions, "Chapter 4: Working with Key/Value Pairs" in *Learning Spark* provides a very good introduction. The Spark RDD class makes use of Scala implicits, and the `PairRDDFunctions` will be available on any RDD of type `(K,V)`. For `PairRDDFunctions`, `K` and `V` can be of any type, but for the `OrderedRDDFunctions` (`sortByKey`, `repartitionAndSortWithinPartitions`, `filterByRange`) `K` must have some implicit ordering. Most common types, like the numeric types or strings, have their ordering already defined in Scala. To use a custom type, you may have to define the ordering yourself. Spark uses implicit conversion to convert an RDD that meets the `PairRDD` or `OrderedRDD` requirements from a generic type to the `PairRDD` or `OrderedRDD` type. This implicit conversion requires that the correct library already be imported. Thus, to use Spark's `pairRDDFunctions`, you need to have imported the `SparkContext`; i.e., imports must include `import org.apache.spark.SparkContext._`.

> **TIP**
>
> When writing a function that uses `OrderedRDDFunctions` of generic key type, you may need to include code defining an implicit val of type `ordering`. In the secondary sort example, which we will discuss in "Secondary Sort and

repartitionAndSortWithinPartitions", we define an ordering of an object called "Panda Keys" as shown in Example 6-3.

Example 6-3. Defining an implicit ordering to work with OrderedRDDFunctions

```
    implicit def orderByLocationAndName[A <: PandaKey]:
Ordering[A] = {
      Ordering.by(pandaKey => (pandaKey.city, pandaKey.zip,
pandaKey.name))
    }

    implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2
```

Actions on Key/Value Pairs

In "Functions on RDDs: Transformations Versus Actions", we discussed how transformations are computed on the Spark executors when an action is called. We also explained that actions usually move data out of the Spark executors either by collecting it to the driver or by writing to stable storage. In general, we advised you to be very cautious about actions that return unbounded data to the driver as they can cause out-of-memory errors in the driver. Most key/value actions (including `countByKey`, `countByValue`, `lookUp`, and `collectAsMap`) return data to the driver. In most instances they return unbounded data since the number of keys and the number of values are unknown. For example, `countByKey` returns a data point for each key, and thus it may cause memory errors if there are more distinct keys than fit in memory on the driver. Conversely, `lookUp` returns all the values for each key, so it will cause memory problems if one key has more data than will fit in memory on the driver.

NOTE

> The `lookup` operation is also expensive because it triggers a shuffle if the RDD doesn't have a known partitioner.

In addition to number of records, the size of each record is an important factor in causing memory errors. For example, if each record is a custom object or a collection type, an action that succeeded in collecting the same number of records in an RDD of bytes may still fail. In general, we want to try to design key/value problems so that the keys fit into memory on the driver. The values should be at least well distributed by key and at best distributed so that each key has no more records than can fit in memory on each executor. We will discuss the effects of bad key distribution in "Straggler Detection and Unbalanced Data", and will provide some suggestions for working around skewed data. As with all Spark programs, we should try to perform transformations that reduce the size of the data before calling actions that move results to the driver.

Key/value transformations can also cause memory errors, most often in the executors, if they require all the data associated with one key to be kept in memory on one partition. Avoiding memory errors and optimizing transformations for fewer shuffles is a bit more complicated than avoiding problems with actions. Thus, key/value transformations will be the focus of the rest of this chapter.

What's So Dangerous About the groupByKey Function

Many sources—including the Spark documentation—warn against the scalability of the `groupByKey` function, which returns an iterator of each element by key. This section attempts to explain the cases in which

`groupByKey` causes problems at scale. We also hope to offer and some advice about alternatives to using `groupByKey`. First, we want to revisit the Goldilocks case, because our first solution to the Goldilocks problem was to use `groupByKey`.

Goldilocks Version 1: groupByKey Solution

One simple solution to the Goldilocks problem is to use `groupByKey` to group the element in each column. `GroupByKey` returns an iterator of elements by each key, so to sort the elements by key we have to convert the iterator to an array and then sort the array.[2] After converting the iterator to an array, we can sort the array and filter for the elements that correspond to our rank statistics.

Example 6-4 is an implementation of the `groupByKey` solution. For consistency, this function also takes a `DataFrame` and a list of element positions as `long` values. It calls the function that creates key/value pairs that we described in Example 6-2.

Example 6-4. Goldilocks version 1, GroupByKey solution

```
  def findRankStatistics(
    dataFrame: DataFrame,
    ranks: List[Long]): Map[Int, Iterable[Double]] = {
    require(ranks.forall(_ > 0))
    //Map to column index, value pairs
    val pairRDD: RDD[(Int, Double)] =
mapToKeyValuePairs(dataFrame)

    val groupColumns: RDD[(Int, Iterable[Double])] =
pairRDD.groupByKey()
    groupColumns.mapValues(
      iter => {
        //convert to an array and sort
        val sortedIter = iter.toArray.sorted

        sortedIter.toIterable.zipWithIndex.flatMap({
        case (colValue, index) =>
```